

ICS Exercise — Week 8

May 8, 2026

Problem 1: I/O Tracing with `fork` and `dup2`

Before the program below is launched, the filesystem contains:

- `/work/notes.txt` — exactly the 5 bytes `ABCDE` (no trailing newline).
- `/work/log.txt` — exactly the 2 bytes `xy` (no trailing newline).
- `/work/out.txt` — does not exist.

The program is launched as `./iotrace` from a shell, with stdout connected to a terminal (no shell redirection). `Open`, `Read`, `Write`, `Fork`, `Wait`, and `Dup2` are the CSAPP error-checking wrappers; they do not change the semantics of the underlying calls.

```
1  #include "csapp.h"
2
3  int main(void) {
4      int fd3, fd4, fd5;
5      char buf[8];
6      pid_t pid;
7
8      fd3 = Open("/work/notes.txt", O_RDONLY, 0);
9      fd4 = Open("/work/log.txt", O_RDWR | O_APPEND, 0);
10     fd5 = Open("/work/out.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
11
12     Read(fd3, buf, 2);           /* parent: read 2 from notes.txt */
13     Write(fd5, buf, 2);          /* parent: write 2 to out.txt */
14     Write(fd4, "1", 1);          /* parent: append "1" to log.txt */
15     printf("p1");               /* parent: into stdout user buffer */
16
17     pid = Fork();
18     if (pid == 0) {
19         Read(fd3, buf, 2);        /* child: read 2 more from notes */
20         Write(fd5, buf, 2);        /* child: write 2 to out.txt */
21         Dup2(fd4, STDOUT_FILENO); /* child: redirect fd 1 -> log */
22         printf("c1");             /* child: into stdout user buffer */
23         Write(fd5, "Z", 1);        /* child: write 'Z' to out.txt */
24         exit(0);                  /* flushes child stdio buffers */
25     }
26
27     Wait(NULL);
28     Read(fd3, buf, 1);           /* parent: read 1 more from notes */
29     buf[1] = '\0';
30     Write(fd4, "2", 1);          /* parent: append "2" to log.txt */
31     printf("p2:%s\n", buf);      /* parent: line ends with '\n' */
32     return 0;
33 }
```

1.1 Final on-disk file contents

After the program finishes (the parent has returned 0 from `main`), give the **exact** final byte contents of each of `/work/notes.txt`, `/work/log.txt`, and `/work/out.txt`. Show the byte sequences inside quotes; do not include any quote character or trailing newline that is not actually present in the file.

File	Final contents	Length
<code>/work/notes.txt</code>	<code>[a]</code>	<code>[b]</code>
<code>/work/log.txt</code>	<code>[c]</code>	<code>[d]</code>
<code>/work/out.txt</code>	<code>[e]</code>	<code>[f]</code>

1.2 Stdout (terminal) output

Write the **exact** sequence of bytes that arrive on the terminal — that is, on the parent's original stdout. Order matters; treat every `\n` as one byte and every printable character as one byte.

1.3 Three-table-model diagnostic

- a. Immediately after line 21 finishes executing in the child (so right after `Dup2(fd4, STDOUT_FILENO)` returns but before `printf("c1")` on line 22), what is the **reference count of the system-wide open file table entry that backs `/work/log.txt`**? List every descriptor-table slot (across both processes) that points to that entry. Justify in 2–3 sentences using the three-table model from lecture.
- b. The parent's `Read(fd3, buf, 1)` on line 28 reads byte `'E'`, **not** byte `'C'`. Explain in 2–3 sentences **where the file offset for `/work/notes.txt` lives** in the kernel data structures, and why fork sharing makes the child's earlier read on line 19 visible to the parent's read on line 28. Your answer must distinguish between the descriptor table, the open file table, and the v-node table.

Problem 2: Robust I/O on a TCP echo server

The function `echo_stdio` below is supposed to read each line a network client sends and echo it back over the same TCP connection. The descriptor `connfd` is a connected stream socket.

```
1  #include "csapp.h"
2
3  void echo_stdio(int connfd) {
4      FILE *stream = fdopen(connfd, "r+");
5      char line[MAXLINE];
6      while (fgets(line, MAXLINE, stream) != NULL) {
7          fputs(line, stream);
8      }
9      fclose(stream);
10 }
```

2.1 Bug hunt

`echo_stdio` works fine when `connfd` is a regular file but is broken in **at least two distinct ways** when `connfd` is a TCP socket. Identify two such defects. For each one:

- Name the relevant concept from the lecture (e.g., "stream-direction switching", "full output buffering", "async-signal safety", "non-seeking descriptor").
- In 1–2 sentences, explain why the defect specifically bites *on a socket* — i.e., what makes a socket different from a regular file with respect to that concept.

2.2 Rewrite with RIO

Rewrite `echo_stdio` using the **buffered-input** RIO API and the **unbuffered-output** `rio_writen`. Your rewrite must:

- call `rio_readinitb` exactly once,
- use `rio_readlineb` to read each line,
- use `rio_writen` to write each line back,
- loop until `rio_readlineb` returns 0 (the client closed the connection),
- use no Standard I/O.

Write the body of the new `void echo_rio(int connfd)`.

2.3 `rio_readlineb` buffer trace

The server has called `rio_readinitb(&rio, connfd)` and then makes successive `rio_readlineb(&rio, buf, 100)` calls. Treat `RIO_BUFSIZE` as 8192 (so the kernel's available bytes — not the 8192-byte buffer — are always the limiting factor on each refill).

The kernel delivers the byte stream `HEL\nWORLD\nFOO\nBAR\nBAZ\n` (22 bytes total — every `\n` is one literal newline) to the application via the following sequence of `read()` syscall returns. Each row gives the bytes returned by the next call to `read()` that `rio_read` makes in order to refill the internal buffer.

<code>read()</code> call #	Bytes returned (count, content)
1	6, <code>HEL\nWO</code>
2	4, <code>RLD\n</code>
3	9, <code>FOO\nBAR\nB</code>
4	2, <code>AZ</code>
5	1, <code>\n</code>
6	0 (EOF)

Fill in the table below. For each successive `rio_readlineb` call:

- **Returned `buf`** — the C string (excluding the trailing null terminator) that the call writes into `buf`.
- **Return value** — the integer that `rio_readlineb` returns.
- **`rio_cnt` after** — the value of `rp->rio_cnt` *after* the call returns.
- **`bufptr` offset after** — the offset of `rp->bufptr` from `&rp->rio_buf[0]` *after* the call returns.
- **# of refills inside this call** — how many times `rio_read` invoked the underlying `read()` syscall during this call.

Initial state (before Call 1): `rio_cnt = 0`, `bufptr` offset = 0.

Call	Returned buf	Return	rio_cnt after	bufptr offset after	# refills
1	[a1]	[a2]	[a3]	[a4]	[a5]
2	[b1]	[b2]	[b3]	[b4]	[b5]
3	[c1]	[c2]	[c3]	[c4]	[c5]
4	[d1]	[d2]	[d3]	[d4]	[d5]
5	[e1]	[e2]	[e3]	[e4]	[e5]
6	[f1]	[f2]	[f3]	[f4]	[f5]

Hint. Recall the canonical CSAPP `rio_read` loop: if `rio_cnt <= 0`, refill the internal buffer with `read(rio_fd, rio_buf, RIO_BUFSIZE)` and reset `rio_bufptr = rio_buf`; otherwise, copy `min(n, rio_cnt)` bytes from `rio_bufptr` to the user buffer, advance `rio_bufptr`, and decrement `rio_cnt`.